

Report for Assignment 3

Group 19

Software Engineering Fundamentals (DD2440)

February 20, 2026

Project.....	1
1. Onboarding Experience.....	1
2. Complexity.....	1
2.1 RequestHandler.handle.....	2
2.2 MultiPartBuilder.part.....	2
2.3 HttpRequestBuilder.buildInternal.....	2
2.4 MockHandler.handle.....	2
2.5 ScenarioEngine.fork.....	3
3. Refactoring.....	3
3.1 RequestHandler.handle.....	3
3.2 MultiPartBuilder.part.....	4
3.3 HttpRequestBuilder.buildInternal.....	4
3.4 MockHandler.handle.....	4
3.5 ScenarioEngine.fork.....	5
4. Coverage.....	5
4.1 Tools.....	5
4.2 Our own coverage tool.....	5
4.3 Evaluation.....	5
5. Coverage Improvement.....	6
5.1 RequestHandler.handle.....	6
5.2 MultiPartBuilder.part.....	6
5.3 HttpRequestBuilder.buildInternal.....	6
5.4 MockHandler.handle.....	7
5.5 ScenarioEngine.fork.....	7
6. Self-assessment: Way of working.....	7
7. Overall Experience.....	8
8. Statement of Contributions:.....	8

Project

Name: Karate

URL: <https://github.com/karatelabs/karate/tree/v1.5.2>

Karate is an open-source testing framework that unifies API, UI, and performance testing in a single, readable format. It lets you write readable tests in a simple Gherkin-style and run them easily in Java projects.

1. Onboarding Experience

Building the Karate project was fairly straightforward: the main requirements were Maven and a compatible Java version ($\geq$ JDK 17), although additional tools (Lizard and JaCoCo) were needed for complexity and coverage analysis.

The tools are well documented, and Maven automatically downloaded the required dependencies and plugins during the build (using `mvn install`). The build did not succeed immediately due to a Java version mismatch in the environment, but it worked after fixing this. Tests generally run well, but coverage reporting was somewhat difficult to set up and was sometimes skipped due to missing execution data, since the parent project does not produce a combined report, and coverage had to be generated separately for the individual modules containing tests. We plan to continue this project.

2. Complexity

In the manual count of CC, we include if-statements, logical operators, catch-blocks (exceptions), loops, and ternary operators. The table below shows the metrics for the different functions.

Function name	CC (lizard count)	CC (manual count)	NLOC
RequestHandler.handle	22	22	68
MultiPartBuilder.part	21	21	82
HttpRequestBuilder.buildInternal	26	26	67
MockHandler.handle	23	23	85
ScenarioEngine.fork	12	12	47

2.1 RequestHandler.handle

This function is ranked in the top 20 for both cyclomatic complexity (CC) and NLOC in the project, meaning it is not only complex but also relatively long.

The purpose of this function is to handle incoming HTTP requests: it normalizes the request path, serves static resources when possible, resolves or creates a session, and then delegates further processing to RequestCycle. The function contains no exception handling, so exceptions do not contribute to its measured complexity. Finally, the method lacks Javadoc or inline comments, which makes it harder to read and understand given its size and number of branches.

2.2 MultiPartBuilder.part

The purpose of this function is to add a “part” to an HTTP request body. This is done in two different ways, depending on whether the request is multipart or not:

1. **Non-multipart mode.** The function builds a URL-encoded form body. The value is normalized into a list of strings. Then, each string is added as a form field.
2. **Multipart.** Depending on what `value` is, the function either:
 - a. adds it as a file,
 - b. recursively calls itself until all values are added, or
 - c. resolves a resource-type from the content-type (if explicitly defined; otherwise, tries to infer it).

This function is longer than complex, but there is a lot of potential for improving the readability of the code. Additionally, it lacks documentation, which makes it difficult to understand its purpose without having to dig into its body.

2.3 HttpRequestBuilder.buildInternal

The purpose of `buildInternal` is to prepare an HTTP request before it is sent by ensuring that the URL method is set, handling multipart data, converting form fields into parameters when necessary, and finally setting the correct content type and character set. The method does not have a Javadoc comment, which makes it slightly harder to understand at first glance, although the code is mostly readable on its own. A few inline comments are present, but they are limited and not very descriptive.

2.4 MockHandler.handle

The purpose of the `MockHandler` class is to turn Karate feature files into an HTTP mock server, and the `handle` method specifically processes incoming HTTP requests, finds a matching scenario, executes it, and returns the generated HTTP response.

2.5 ScenarioEngine.fork

The purpose of the ScenarioEngine.fork method is to start and manage an external system process from a Karate scenario. It prepares the command to execute, applies optional configuration such as shell usage, environment variables, working directory, stream redirection, and output listeners, and then creates a command object. By default, the process is started immediately, but execution can be deferred via an option. The method contains many conditional branches due to the number of supported options and lacks documentation, which negatively impacts readability despite its relatively straightforward logic.

3. Refactoring

The table below indicates CC (lizard count), CC reduction in percentage, and NLOC after carrying out the refactoring, as well as links to the functions before and after refactoring.

Function name	CC	CC reduction	NLOC	Before	After
RequestHandler.handle	5	77%	19	Link	Link
MultiPartBuilder.part	5	76%	16	Link	Link
HttpRequestBuilder.buildInternal	1	96%	8	Link	Link
MockHandler.handle	6	74%	29	Link	Link
ScenarioEngine.fork	1	92%	11	Link	Link

3.1 RequestHandler.handle

This function serves a lot of purposes, such as stripping context path, handling sessions, and redirection. There are also a lot of conditionals, many of them nested inside others, which makes the function hard to read. To reduce complexity, different parts of the function can be split into smaller functions. For example, stripping the context path can be made into its own helper function stripContextPath. Similarly, parts of the function, like resolving the home page path, setting resource type, serving static resources, etc., can all be moved to their own helper functions. By doing this, the handle method becomes clearer and easier to read, while its cyclomatic complexity gets reduced drastically thanks to the lower number of conditionals. Furthermore, this refactoring also makes the code easier to test, since each part can be tested independently. However, this refactoring would lead to the code being more “spread out” across the class, which could make it harder to navigate.

3.2 MultiPartBuilder.part

Since this function acts differently depending on what the value-variable is, the four bodies of the if-else statements can be moved to separate functions. This also makes sense because the four branches do not interact in any way; the reason the original author put it all together is most likely because it is convenient.

Furthermore, the instructions for resolving the char set and creating a list from the input value can also be placed in separate functions. This even makes the code more reusable in the future, since those instructions are more generic (i.e., they do not really depend on the current class). One possible drawback to this is that the code could get more difficult to understand if you want to dig deep into the source, because you need to jump around the different functions. But, on the other hand, it is much more maintainable because, instead of having to read through an entire page's worth of a function, we only see function calls (with appropriate names, so it is not difficult to grasp what is happening).

3.3 HttpRequestBuilder.buildInternal

My refactoring plan would be to extract the guard clauses into private helpers, for instance, `resolveUrl()`, `resolveMethod()`, `convertGetFormFields()`, `prepareMultipartBody()`, `applyCookiesHeader()`, and `inferContentType()`, and then use early returns to flatten nesting. Then I would add unit tests for each helper and one integration test for `buildInternal`. This all would in turn reduce the CC (Cyclomatic Complexity) of the `buildInternal` method/function, and because the code was so isolated, the different helpers are not dependent on each other, leading to low CC for all helpers. Note: I carried out this refactoring process (results can be seen beneath the Complexity section), but without adding any additional tests because the focus of the assignment was on reducing the CC for the chosen function.

3.4 MockHandler.handle

This method does a variety of things all in service of processing an incoming HTTP request, finding a matching scenario, executing it, and returning the generated HTTP response. There are a lot of conditionals and loops to make this happen, and the method, which also lacks any documentation, is therefore pretty hard to understand. My goal with the refactoring was therefore to make it more understandable and manageable if for instance future changes needed to be implemented. To do that, I split the method into multiple methods where it felt natural and necessary to make it more comprehensible. Introducing four helper methods and better documentation made it more understandable, flexible for future changes, and reduced the CC down from 23 to 6.

3.5 ScenarioEngine.fork

The original fork method contained many conditional branches to handle the different optional parameters used to configure and start an external process. Although the logic itself was not complex, the accumulation of conditionals negatively impacted readability and cyclomatic complexity. The refactoring splits the method into several small helper functions, each responsible for a single concern such as resolving command arguments, configuring the environment, or starting the process. This significantly reduces the cyclomatic complexity of the main fork method and makes the code easier to read, test, and maintain, at the cost of slightly increased code dispersion across the class.

4. Coverage

4.1 Tools

We used JaCoCo (Java Code Coverage Library) for measuring branch coverage. Because the tool is well-known and widely used, there is good documentation, and it was easy to integrate. We only needed to add the “plugin code” into the pom.xml and download and make sure that it runs. After that, it generates reports of the coverage for the wanted methods. The actual usage came in handy and was favorable because it generated clean reports of the coverage.

4.2 Our own coverage tool

[Link to diff with our own coverage tool](#)

4.3 Evaluation

The quality of our coverage measurement is block level; that is, it handles if/else/for-loops, etc., but doesn’t handle any short-circuit logic (&&, ||, etc) like lizard or JaCoCo. The tool can also handle ternary operators.

When it comes to limitations of our tool, scalability is a big one. A tool like this works for five functions, but if there were thousands, manually setting flags would be impossible. It’s also not that good at adapting. Say we added an if statement at the top of the method, then we would manually have to change all the indexes for the rest of the flags. All changes must be manually updated.

The results from our tool are reasonably consistent with existing coverage tools. For example, for MockHandler.handle, JaCoCo reports 44 branches, and that the tests cover 32 of them, while our manual tool reports 34 branches with 21 covered. This corresponds to 60% branch coverage according to my tool, compared to 72% reported by JaCoCo. Although the numbers

differ, the overall conclusion is similar, and the differences can likely be explained by earlier limitations, such as not counting short-circuit logic and other edge cases in branch detection.

5. Coverage Improvement

5.1 RequestHandler.handle

[Link to added tests](#) (6 new tests)

Old coverage:

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
● handle(Request)		42 %		42 %	16	22	23	45	0	1

New coverage:

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
● handle(Request)		70 %		69 %	10	22	10	45	0	1

5.2 MultiPartBuilder.part

[Link to added tests](#) (4 new tests)

Old coverage:

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
● part(Map)		70 %		62 %	13	21	28	94	0	1

New coverage:

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
● part(Map)		92 %		88 %	4	19	5	59	0	1

5.3 HttpRequestBuilder.buildInternal

[Link to added tests](#) (4 new tests)

- Before adding new tests (branch coverage):
BRANCH: 35/50 (70%) | missed=15, covered=35.
- After adding new tests (branch coverage):
BRANCH: 44/50 (88%) | missed=6, covered=44.

5.4 MockHandler.handle

[Link to added tests](#) (4 new tests)

Old coverage:

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
● handle(Request)		69 %		72 %	10	23	14	64	0	1

Before adding new tests (branch coverage): BRANCH: 32/44 (72%) | missed=12, covered=32.

New coverage:

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
● handle(Request)		91 %		86 %	6	23	3	64	0	1

After adding new tests (branch coverage): BRANCH: 38/44 (86%) | missed=6, covered=38.

5.5 ScenarioEngine.fork

[Link to added tests](#) (4 new tests)

Old coverage:

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
● fork(boolean, Map)		75%		50%	11	12	6	34	0	1

New coverage:

● fork(boolean, Map)		84%		72%	6	12	3	34	0	1
--------------------------------------	------------------------	-----	------------------------	-----	---	----	---	----	---	---

6. Self-assessment: Way of working

Based on the Essence checklist, we would say that our way of working is still in the In Place state, but closer to Working Well than earlier in the course. Everyone follows the agreed practices and tools, and we all have access to the resources needed to do the work. During the course, we have improved in how we collaborate, for example, by adopting certain conventions and communicating clearly about our progress.

We also discuss our workflow openly and adjust it when needed. However, we don't consider ourselves to have fully reached the Working Well state yet, mainly because the course runs in parallel with other responsibilities, leaving us with limited time to complete the assignments. This often requires us to split the work and focus on separate parts, making it harder to get a full overview of the system and fully understand each other's changes outside of code reviews. As a result, collaboration sometimes becomes more about combining individual contributions than working seamlessly as a unit.

To improve further, we could spend more time-sharing knowledge within the group, for instance, through short sync-ups or occasional pair programming on key parts of the assignments.

7. Overall Experience

This assignment taught us a lot about code complexity and coverage, and how to work with these metrics. By measuring cyclomatic complexity using lizard, we gained a better understanding of how complexity arises from factors such as a high number of branches and the overall length of a function. Comparing lizard's results with our manual counts showed that cyclomatic complexity can vary depending on the method used, while also highlighting the usefulness of automated tools for quickly assessing code complexity.

When analyzing branch coverage, we practiced identifying untested paths and writing test cases to cover them, improving the coverage of the five functions we focused on. This process also demonstrated the value of automated coverage tools such as JaCoCo, which efficiently highlight gaps that might otherwise be overlooked.

Furthermore, exploring refactoring opportunities in these complex functions helped us understand how code can be split into smaller, more manageable units and how doing so reduces complexity, makes the code easier to read and maintain, and simplifies testing.

8. Statement of Contributions:

Elias Gaghlasian:

- Implemented 4 tests for branch coverage and refactoring **HttpRequestBuilder.buildInternal**.

Elias Hollstrand :

- Implemented 6 tests for branch coverage and refactoring **RequestHandler.handle**. Also set up for the JaCoCo tool.

Jonathan Skantz:

- Implemented 4 tests for branch coverage and refactoring **MultiPartBuilder.part**.

Fabian Holm:

- Implemented 4 tests for branch coverage, refactoring, and DIY branch coverage tool **MockHandler.handle**.

Vadim El Guedj:

- Implemented 4 tests for branch coverage and refactoring **ScenarioEngine.fork**.